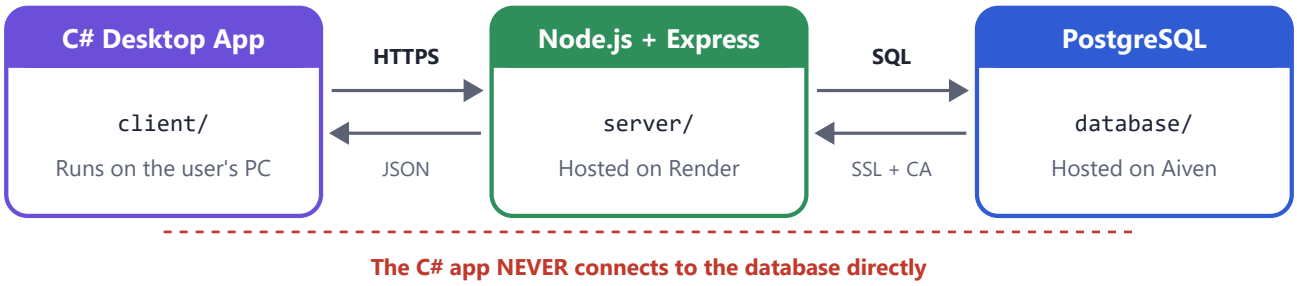
AGP INVENTORY SYSTEM · BEGINNER'S GUIDE

# Building Your First Full-Stack App

C# desktop frontend · Node.js + Express API on Render · PostgreSQL on Aiven




A step-by-step plan: what to install, which accounts to create, how the repository is organised, how to build and deploy each part, which C# UI libraries to use, and how to work as a team.

Version 1.0 · November 2026

# Contents

1. Read This First . . . 3
2. The Big Picture . . . 4
3. Tools and Accounts . . . 6
4. The Repository . . . 8
5. Project Plan and Timeline . . . 9
6. Phase 1 – Database on Aiven . . . 11
7. Phase 2 – Build the Express API . . . 14
8. Phase 3 – Deploy the API on Render . . . 19
9. Phase 4 – Create the C# App and Design the UI . . . 21
10. Phase 5 – Connect the C# App to the API . . . 23
11. UI Libraries for C# . . . 26
12. Working as a Team with Git and GitHub . . . 28
13. Security Basics . . . 29
14. Next Step: Adding Login (bcrypt + JWT) . . . 30
15. Troubleshooting . . . 31
16. Final Checklist . . . 32
17. Glossary and Useful Links . . . 33

SECTION 1

# Read This First

This guide is written for people who have never built a full application before. Follow it in order. Every phase ends with a checkpoint, so you always know whether you can move on.

## What you will build

A desktop application written in C#. When the user clicks a button, the app sends a request over the internet to a small web server (the **API**) written in JavaScript with Node.js and Express. The API reads or writes data in a **PostgreSQL** database and sends the answer back as JSON. The API is hosted on **Render** and the database on **Aiven**, so both are online 24/7 and every team member uses the same data.

Throughout the guide we use a tiny practice feature, a **task list** (table `tasks`), so you can learn the full flow end to end. Once it works, replace it with your real project's features (users, products, orders and so on) using exactly the same pattern. For the AGP Inventory System that means tables such as `products`, `categories`, `suppliers`, `stock_movements` and `users`.

## How to use this guide

- Do the phases **in order**: Database, then API, then Deploy, then C# app, then Connect. Each part depends on the one before.
- Type the code yourself instead of copy-pasting. You will understand it and remember it.
- When something breaks, check **Section 15 – Troubleshooting** before asking for help.
- Commit your work to Git at the end of every working session.

## The five golden rules

| # | Rule | Why |
|---|---|---|
| 1 | The C# app never talks to the database directly. | If the database password were inside the .exe, anyone could extract it and delete your data. |
| 2 | Secrets live in `.env` / Render environment variables, never in code or Git. | GitHub repositories are easy to leak. Bots scan them for passwords within minutes. |
| 3 | Always use parameterised SQL (`$1, $2`), never string concatenation. | Prevents SQL injection, the most common way databases get hacked. |
| 4 | Never store plain-text passwords. Hash them with bcrypt. | If the database leaks, users' real passwords stay safe. |
| 5 | Small steps, test often, commit often. | When something breaks you only have a little code to check. |

SECTION 2

# The Big Picture

Before writing any code, understand what each piece does and how a single click travels through the whole system.

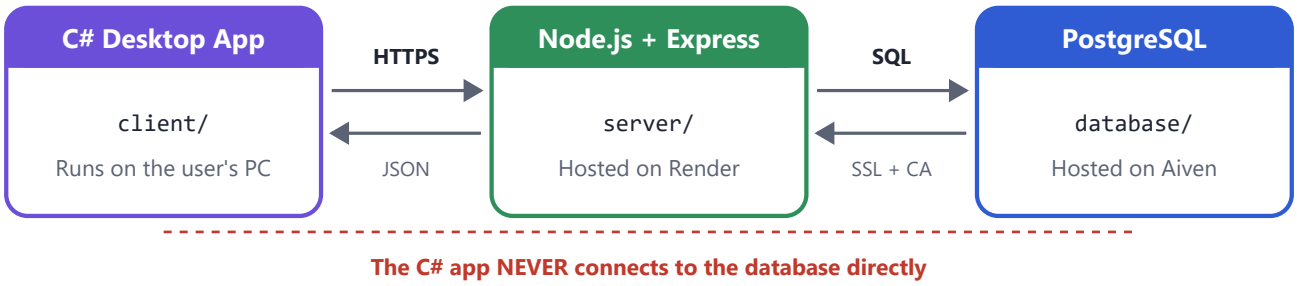



*Figure 1 – The three layers and how they communicate*

| Layer | Technology | Its job | Where it runs |
|---|---|---|---|
| Frontend (client) | C# with .NET – WinForms or WPF | Shows screens, collects input, displays results. Contains no passwords and no SQL. | On each user's Windows PC |
| Backend (API) | Node.js + Express | Receives HTTP requests, checks them, runs SQL, returns JSON. Holds all the business rules. | Render (cloud) |
| Database | PostgreSQL | Stores data permanently in tables. | Aiven (cloud) |

## One click, step by step

Example: the user types *Buy milk* and clicks **Add Task**.

1. The C# button handler calls `ApiClient.CreateTaskAsync("Buy milk")`.
2. ApiClient sends `POST https://your-app.onrender.com/api/tasks` with the body `{ "title": "Buy milk" }`.
3. Express matches the URL to a **route**, which calls the **controller** function `createTask`.
4. The controller checks the input, then the **model** runs `INSERT INTO tasks (title) VALUES ($1) RETURNING *`.
5. PostgreSQL on Aiven saves the row and returns it (with its new id).
6. Express replies with status `201 Created` and the new task as JSON.
7. The C# app turns the JSON into a TaskItem object and adds it to the grid.

## Key words you will see a lot

| Term | Meaning |
|---|---|
| API | Application Programming Interface: a set of URLs your app can call to get or change data. |
| Endpoint | One specific URL + method, e.g. `GET /api/tasks`. |
| HTTP method | The action: `GET` read, `POST` create, `PUT`/`PATCH` update, `DELETE` remove. |
| JSON | Text format for data: `{ "id": 1, "title": "Buy milk" }`. |
| Status code | Number telling the result: `200` OK, `201` created, `400` bad input, `401` not logged in, `404` not found, `500` server error. |
| Environment variable | A setting given to the program from outside (passwords, URLs) so it is not written in the code. |
| Deploy | Put your code on a server on the internet so others can use it. |

SECTION 3

# Tools and Accounts

Install everything in this section before Phase 1. Every team member needs the tools; only one person needs to create the Render and Aiven services (then invite the others).

## Software to install

| Tool | Used for | Get it from | Check it works |
|---|---|---|---|
| Git | Version control, sharing code | git-scm.com | `git --version` |
| Node.js (LTS version) | Running the Express server | nodejs.org | `node -v` and `npm -v` |
| Visual Studio Community (2022 or newer) | Writing the C# app. Tick the **.NET desktop development** workload. | visualstudio.microsoft.com | Can create a "Windows Forms App" project |
| .NET SDK (LTS) | Building C# (usually installed with Visual Studio) | dotnet.microsoft.com | `dotnet --version` |
| VS Code | Editing the Node.js server | code.visualstudio.com | Opens the server folder |
| DBeaver Community or pgAdmin | Viewing tables and running SQL on Aiven | dbeaver.io / pgadmin.org | Connects to the Aiven database |
| Postman, Bruno or Thunder Client (VS Code extension) | Testing API endpoints without the C# app | postman.com / usebruno.com / VS Code marketplace | Can send a GET request |

## Accounts to create

| Service | Why | Notes |
|---|---|---|
| GitHub (github.com) | Stores the repository; Render deploys from it. | One person creates the repo and adds teammates as collaborators. |
| Render (render.com) | Hosts the Express API. | Sign up **with GitHub** so Render can see your repositories. The free plan is enough for school projects. |
| Aiven (aiven.io) | Hosts PostgreSQL. | Pick the free PostgreSQL plan. Invite teammates to the project so they can see connection details. |

**TIP** Use a shared team email or add every member to the Render and Aiven projects, so the project does not die if one person loses access.

## Recommended VS Code extensions

- ESLint (finds JavaScript mistakes)
- Prettier (auto-formats code)
- Thunder Client (test the API inside VS Code)
- DotENV (colours .env files)

**CHECKPOINT** Run all the “Check it works” commands in a new terminal. If any of them prints an error, fix the installation before continuing.

SECTION 4

# The Repository

The repository is already prepared with an empty folder structure and a README in each main folder. Here is what everything is for.

**Folder structure**

```
AGP-Inventory-System/
├── README.md              overview + quick links
├── .gitignore             files Git must never upload
├── client/                C# frontend
│   ├── README.md
│   └── AppClient/
│       ├── Forms/         screens / windows
│       ├── Models/        classes that match the API's JSON
│       ├── Services/      ApiClient - all HTTP calls
│       ├── Helpers/       AppConfig, validation, formatting
│       └── Assets/        images/, icons/
├── server/                Node.js + Express backend
│   ├── README.md
│   ├── .env.example       template for your .env
│   ├── certs/             Aiven ca.pem (ignored by Git)
│   └── src/
│       ├── config/        db.js - database connection
│       ├── routes/        URL -> controller
│       ├── controllers/   request logic
│       ├── models/        SQL queries
│       ├── middleware/    auth, error handler
│       └── utils/         helpers
├── database/
│   ├── README.md
│   ├── migrations/        001_..., 002_... CREATE TABLE scripts
│   ├── seeds/             sample data
│   └── diagrams/          ERD
└── docs/
    ├── Beginners-Guide.pdf
    └── UI-LIBRARIES.md
```

Empty folders contain a file called `.gitkeep`. Git does not store empty folders, so this placeholder keeps them in the repository. You can delete it once the folder has real files.

## Getting the repository (every member)

The repository is hosted at **github.com/Shaloh69/AGP-Inventory-System**. Accept the collaborator invitation from your GitHub email, then:

**Terminal**

```
git clone https://github.com/Shaloh69/AGP-Inventory-System.git
cd AGP-Inventory-System
```

**WATCH OUT** `.gitignore` already blocks `.env`, `*.pem`, `node_modules/`, `bin/` and `obj/`. Run `git status` before every commit and make sure none of these appear.

SECTION 5

# Project Plan and Timeline

A suggested 8-week plan for a small team (3–5 people). Shrink or stretch the weeks to fit your deadline, but keep the order.

| Phase | Wk | Goal | Main tasks | Checkpoint (done when…) |
|---|---|---|---|---|
| 0 · Setup | 1 | Everyone ready | Install tools, create accounts, clone repo, agree on features | Everyone can run git, node and dotnet |
| Plan | 1 | Know what to build | List features, draw screen sketches, draw the ERD, list API endpoints | Team agrees on a one-page feature list |
| 1 · Database | 2 | Tables online | Create Aiven service, write migrations, add seed data | You can SELECT the seed data in DBeaver |
| 2 · API | 3–4 | Working endpoints locally | Express setup, db.js, one route/controller/model per table | All endpoints work in Postman on localhost |
| 3 · Deploy | 4 | API online | Create Render web service, set env vars + secret file | Health URL works on onrender.com |
| 4 · C# UI | 3–5 | Screens designed | Create project, install UI library, build forms with fake data | Every screen opens and navigates |
| 5 · Connect | 5–6 | Real data in the app | ApiClient, models, replace fake data with API calls | Data created in the app appears in Aiven |
| 6 · Extras | 6–7 | Login and polish | bcrypt + JWT login, validation, loading/error messages | Wrong password is rejected; nothing crashes |
| 7 · Finish | 8 | Ready to submit | Testing, bug fixes, README/API docs, demo rehearsal | Final checklist (Section 16) complete |

Phases 2 and 4 can run **at the same time**: backend members build the API while frontend members design screens with fake data.

## Suggested team roles

| Role | Responsible for | Folders |
|---|---|---|
| Project lead | Plan, GitHub repo, reviewing pull requests, Render/Aiven accounts | everything |
| Database | ERD, migrations, seeds, SQL help | database/ |
| Backend | Express routes, controllers, models, deployment | server/ |
| Frontend (1–2 people) | C# forms, UI library, ApiClient | client/ |
| QA / docs | Testing every feature, writing docs, preparing the demo | docs/ |

## Planning outputs (do these before coding)

- [ ] **Feature list** – what the user can do, e.g. “register”, “log in”, “add a task”, “mark a task done”.
- [ ] **Screen sketches** – one box per form, drawn on paper or in Figma / Excalidraw.
- [ ] **ERD** – tables, columns and relationships (save the image in database/diagrams/).
- [ ] **Endpoint list** – for each feature: method, URL, what it receives, what it returns (save as `docs/API.md`).

SECTION 6

# Phase 1 – Database on Aiven

Create the PostgreSQL database first, because the API needs it and the API's shape depends on your tables.

## Step 1 – Create the service

1. Log in to **console.aiven.io** and open (or create) a project.
2. Click **Create service** and choose **PostgreSQL**.
3. Choose the **Free** plan (or a trial plan) and a cloud region. Pick the region **closest to the Render region** you will use; it makes the app faster.
4. Give the service a name (e.g. *agp-db*) and click **Create**. Wait until the status changes to **Running** (a few minutes).

## Step 2 – Collect the connection details

Open the service's **Overview** page. You need:

| Item | Where it goes |
|---|---|
| Service URI (postgres://avnadmin:...@...aivencloud.com:PORT/defaultdb?sslmode=require) | `server/.env` as `DATABASE_URL` – **delete** the `?sslmode=require` part |
| CA certificate (Download button) | Save as `server/certs/ca.pem` |
| Host, Port, User, Password, Database name | DBeaver / pgAdmin connection |

**WATCH OUT** These details are secret. Share them with your team through a private chat, never in GitHub, screenshots or public group chats.

## Step 3 – Connect with DBeaver

1. DBeaver → **New Database Connection** → **PostgreSQL**.
2. Fill in Host, Port, Database, Username and Password from Aiven.
3. Open the **SSL** tab → tick **Use SSL** → SSL mode **require** (or **verify-full** and choose your ca.pem as the root certificate).
4. Click **Test Connection**. DBeaver may ask to download the PostgreSQL driver: accept.

## Step 4 – Write your first migration

A migration is a SQL file that creates or changes tables. Save each one in `database/migrations/` with a number so everyone runs them in the same order. Open it in DBeaver's SQL editor and run it.

**database/migrations/001_create_tasks_table.sql**

```
-- database/migrations/001_create_tasks_table.sql
CREATE TABLE tasks (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR(200) NOT NULL,
    is_done     BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
```

**database/seeds/001_sample_tasks.sql**

```
-- database/seeds/001_sample_tasks.sql
INSERT INTO tasks (title) VALUES
    ('Install Node.js'),
    ('Create Aiven database'),
    ('Deploy API to Render');
```

## Example of a relationship (users own tasks)

**Relationship example**

```
-- 002_create_users_table.sql
CREATE TABLE users (
    id             SERIAL PRIMARY KEY,
    username       VARCHAR(50)  NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,      -- never the real password!
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- 003_add_user_id_to_tasks.sql
ALTER TABLE tasks
    ADD COLUMN user_id INTEGER REFERENCES users(id) ON DELETE CASCADE;
```

## Common PostgreSQL data types

| Type | Use for | C# type |
|---|---|---|
| `SERIAL` / `INTEGER` | ids, counts | `int` |
| `VARCHAR(n)` / `TEXT` | names, descriptions | `string` |
| `BOOLEAN` | yes / no flags | `bool` |
| `NUMERIC(10,2)` | money (never use FLOAT for money) | `decimal` |
| `DATE` | birthdays, due dates | `DateTime` / `DateOnly` |
| `TIMESTAMPTZ` | exact moments (created_at) | `DateTime` / `DateTimeOffset` |

**CHECKPOINT** You can see your tables and the three seed rows in DBeaver (SELECT * FROM tasks;). Commit the .sql files to Git.

SECTION 7

# Phase 2 – Build the Express API

Now build the server on your own computer (localhost). Do not deploy until every endpoint works locally.

## Step 1 – Initialise the project

Terminal

```
cd server
npm init -y
npm install express pg dotenv cors
```

| Package | What it does |
|---|---|
| `express` | The web framework: routes, requests, responses. |
| `pg` | The PostgreSQL driver: lets Node.js run SQL. |
| `dotenv` | Loads values from `.env` into `process.env`. |
| `cors` | Allows other origins to call the API (needed later if you add a web front-end; harmless for C#). |

Open `package.json` and replace the `scripts` section. `node --watch` restarts the server automatically when you save a file.

server/package.json (scripts only)

```
"scripts": {
  "start": "node src/index.js",
  "dev": "node --watch src/index.js"
}
```

## Step 2 – Create .env

Copy `.env.example` to `.env` and fill in the values from Aiven. Put the downloaded certificate at `certs/ca.pem`.

server/.env

```
PORT=3000
DATABASE_URL=postgres://avnadmin:YOUR_PASSWORD@YOUR_HOST.aivencloud.com:12345/defaultdb
DB_CA_PATH=./certs/ca.pem
JWT_SECRET=change-me-to-a-long-random-string
```

## Step 3 – Database connection

**server/src/config/db.js**

```
const fs = require('fs');
const { Pool } = require('pg');

// A Pool keeps a few connections open and reuses them (faster than reconnecting).
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true,                        // verify Aiven's certificate
    ca: fs.readFileSync(process.env.DB_CA_PATH).toString(),
  },
});

module.exports = pool;
```

## Step 4 – Model (SQL queries)

**server/src/models/task.model.js**

```
const pool = require('../config/db');

async function findAll() {
  const result = await pool.query('SELECT * FROM tasks ORDER BY id');
  return result.rows;
}

async function findById(id) {
  const result = await pool.query('SELECT * FROM tasks WHERE id = $1', [id]);
  return result.rows[0];                // undefined if not found
}

async function create(title) {
  const result = await pool.query(
    'INSERT INTO tasks (title) VALUES ($1) RETURNING *',
    [title]
  );
  return result.rows[0];
}

async function update(id, title, isDone) {
  const result = await pool.query(
    'UPDATE tasks SET title = $1, is_done = $2 WHERE id = $3 RETURNING *',
    [title, isDone, id]
  );
  return result.rows[0];
}

async function remove(id) {
  const result = await pool.query('DELETE FROM tasks WHERE id = $1', [id]);
  return result.rowCount > 0;
}

module.exports = { findAll, findById, create, update, remove };
```

> **WATCH OUT** See the `$1`, `$2` placeholders? The values are passed separately in the array. **Never** build SQL like `"... WHERE id = " + id`: that allows SQL injection.

## Step 5 – Controller (request logic)

`server/src/controllers/task.controller.js`

```
const Task = require('../models/task.model');

async function getAll(req, res) {
  res.json(await Task.findAll());
}

async function getOne(req, res) {
  const task = await Task.findById(req.params.id);
  if (!task) return res.status(404).json({ message: 'Task not found' });
  res.json(task);
}

async function create(req, res) {
  const { title } = req.body;
  if (!title || title.trim() === '') {
    return res.status(400).json({ message: 'title is required' });
  }
  res.status(201).json(await Task.create(title.trim()));
}

async function update(req, res) {
  const { title, is_done } = req.body;
  const task = await Task.update(req.params.id, title, is_done ?? false);
  if (!task) return res.status(404).json({ message: 'Task not found' });
  res.json(task);
}

async function remove(req, res) {
  const deleted = await Task.remove(req.params.id);
  if (!deleted) return res.status(404).json({ message: 'Task not found' });
  res.status(204).end();
}

module.exports = { getAll, getOne, create, update, remove };
```

## Step 6 – Routes (URL → controller)

`server/src/routes/task.routes.js`

```
const express = require('express');
const controller = require('../controllers/task.controller');

const router = express.Router();

router.get('/', controller.getAll);        // GET    /api/tasks
router.get('/:id', controller.getOne);     // GET    /api/tasks/5
router.post('/', controller.create);       // POST   /api/tasks
router.put('/:id', controller.update);     // PUT    /api/tasks/5
router.delete('/:id', controller.remove);  // DELETE /api/tasks/5

module.exports = router;
```

## Step 7 – Error handler middleware

**server/src/middleware/errorHandler.js**

```
// Express calls this when any route throws an error.
function errorHandler(err, req, res, next) {
  console.error(err);                      // shows in the terminal / Render logs
  res.status(err.status || 500).json({ message: 'Something went wrong on the server' });
}

module.exports = errorHandler;
```

## Step 8 – Entry point

**server/src/index.js**

```
require('dotenv').config();                // must be the first line
const express = require('express');
const cors = require('cors');
const taskRoutes = require('./routes/task.routes');
const errorHandler = require('./middleware/errorHandler');

const app = express();
app.use(cors());
app.use(express.json());                   // lets us read JSON bodies (req.body)

app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
app.use('/api/tasks', taskRoutes);

app.use(errorHandler);                     // always last

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Express 5 (the current major version) automatically sends errors thrown inside **async** route functions to the error handler, so you do not need try/catch in every controller.

## Step 9 – Run and test

```
Terminal
```

```
npm run dev
# Server running on port 3000
```

Open Postman / Bruno / Thunder Client and try each endpoint:

| Method | URL | Body (JSON) | Expected |
|---|---|---|---|
| GET | `http://localhost:3000/api/health` | – | `{ "status": "ok" }` |
| GET | `http://localhost:3000/api/tasks` | – | 200 + array of tasks |
| POST | `http://localhost:3000/api/tasks` | `{ "title": "Test" }` | 201 + the new task |
| POST | `http://localhost:3000/api/tasks` | `{ }` | 400 + message |
| PUT | `http://localhost:3000/api/tasks/1` | `{ "title": "Test", "is_done": true }` | 200 + updated task |
| DELETE | `http://localhost:3000/api/tasks/1` | – | 204 (no body) |
| GET | `http://localhost:3000/api/tasks/9999` | – | 404 + message |

**CHECKPOINT** Every row in the table above gives the expected result, and the changes are visible in DBeaver. Commit and push.

## Adding your real features

For each new table repeat the same four files: `<name>.model.js` → `<name>.controller.js` → `<name>.routes.js` → one `app.use(...)` line in `index.js`.

SECTION 8

# Phase 3 – Deploy the API on Render

Render takes your code from GitHub, runs npm install and npm start, and gives you a public HTTPS address.

## Step 1 – Push your code

Make sure the latest server code is on GitHub (branch `main`). Check that `.env` and `ca.pem` are **not** in the repository.

## Step 2 – Create the Web Service

1. Render dashboard → **New +** → **Web Service** → connect your GitHub repository.
2. Fill in the settings from the table below and choose the **Free** instance type.
3. Before clicking Deploy, open **Advanced** / **Environment** and add the variables and secret file (Step 3).
4. Click **Create Web Service** and watch the logs. The build takes a few minutes.

| Setting | Value |
|---|---|
| Name | e.g. agp-api (becomes agp-api.onrender.com) |
| Region | Same area as your Aiven region |
| Branch | `main` |
| Root Directory | `server` (important: the server is in a sub-folder) |
| Runtime | Node |
| Build Command | `npm install` |
| Start Command | `npm start` |
| Health Check Path (optional) | `/api/health` |

## Step 3 – Environment variables and secret file

| Key | Value |
|---|---|
| `DATABASE_URL` | Same as in your .env (without ?sslmode=require) |
| `DB_CA_PATH` | `/etc/secrets/ca.pem` |
| `JWT_SECRET` | A long random string (only if you add login) |

Then under **Secret Files** add a file named `ca.pem` and paste the full content of the Aiven CA certificate (including the BEGIN/END lines). Render makes it available at `/etc/secrets/ca.pem`. Do **not** add `PORT`; Render sets it for you.

## Step 4 – Test the live API

```
Browser / Postman

https://agp-api.onrender.com/api/health     ->  { "status": "ok" }
https://agp-api.onrender.com/api/tasks      ->  [ ...your tasks... ]
```

## Things to know about Render

- **Auto-deploy:** every push to main redeploys automatically. Broken code on main = broken live API, so test locally first.
- **Logs:** the Logs tab shows your console.log output and errors. This is the first place to look when something fails.
- **Sleeping:** free services sleep after about 15 minutes with no traffic. The next request can take 30–60 seconds while it wakes up. Open the health URL a minute before a demo, and give the C# HttpClient a long timeout.

**CHECKPOINT** The health and tasks URLs work from any computer, using your onrender.com address.

SECTION 9

# Phase 4 – Create the C# App and Design the UI

Build the screens first using fake data. You can do this while the API is still being written.

## Step 1 – WinForms or WPF?

| | Windows Forms (WinForms) | WPF |
|---|---|---|
| Designing screens | Drag and drop in the designer | XAML markup (designer available, but you will write XAML) |
| Learning curve | Easiest – best for first projects | Steeper, more powerful |
| Look | Classic by default; modern with a UI library | Modern styling, animations, templates |
| Recommendation | **Choose this if the team is new to C#** | Choose if someone already knows XAML |

The examples below use **WinForms**. The ApiClient and Models code is identical in WPF.

## Step 2 – Create the project inside client/

```
Terminal (or use Visual Studio: New Project -> Windows Forms App -> folder client)
```

```
cd client
dotnet new sln -n AppClient
dotnet new winforms -n AppClient          # WPF: dotnet new wpf -n AppClient
dotnet sln add AppClient/AppClient.csproj
```

Open `client/AppClient.sln` in Visual Studio. The folders Forms, Models, Services, Helpers and Assets are already there.

## Step 3 – What goes in each folder

| Folder | Contains | Example |
|---|---|---|
| Forms/ | One file per screen. Only UI code: reading text boxes, showing results. | `LoginForm.cs`, `MainForm.cs` |
| Models/ | Plain classes whose properties match the API's JSON. | `TaskItem.cs` |
| Services/ | The only place that uses HttpClient. | `ApiClient.cs` |
| Helpers/ | Settings and small utilities. | `AppConfig.cs`, `Validator.cs` |
| Assets/ | Logos, icons, images (set “Copy to Output Directory” if loaded from disk). | `logo.png` |

## Step 4 – Install a UI library

Right-click the project → **Manage NuGet Packages** → **Browse**, search and install. Or in a terminal:

Terminal

```
cd client/AppClient
dotnet add package MaterialSkin.2
dotnet add package FontAwesome.Sharp
```

Example: turning a normal form into a Material Design form with MaterialSkin.2:

client/AppClient/Forms/MainForm.cs

```
using MaterialSkin;
using MaterialSkin.Controls;

namespace AppClient.Forms;

public partial class MainForm : MaterialForm      // was: Form
{
    public MainForm()
    {
        InitializeComponent();

        var skin = MaterialSkinManager.Instance;
        skin.AddFormToManage(this);
        skin.Theme = MaterialSkinManager.Themes.LIGHT;
        skin.ColorScheme = new ColorScheme(
            Primary.Indigo500, Primary.Indigo700, Primary.Indigo100,
            Accent.Pink200, TextShade.WHITE);
    }
}
```

Section 11 lists many more libraries for WinForms, WPF and cross-platform apps.

## Screen design tips for beginners

- Name every control you use in code: `btnSave`, `txtTitle`, `lblStatus`, `dgvTasks`, `chkDone`.
- Use **Anchor** and **Dock** (or TableLayoutPanel) so the form still looks right when resized.
- Keep one main colour and one accent colour. Use the same font everywhere.
- Always show what is happening: “Loading…”, “Saved!”, or a clear error message.
- Disable a button while its request is running so the user cannot click it twice.
- Start with fake data (a hard-coded List) so you can design without the API.

**CHECKPOINT** Every screen opens, navigates to the others and shows fake data. Commit and push.

SECTION 10

# Phase 5 – Connect the C# App to the API

Replace the fake data with real calls to your Render API.

## Step 1 – One place for the server address

`client/AppClient/Helpers/AppConfig.cs`

```
namespace AppClient.Helpers;

public static class AppConfig
{
    // Local testing:  "http://localhost:3000/"
    // Production:     "https://agp-api.onrender.com/"
    // The URL MUST end with "/".
    public const string ApiBaseUrl = "https://agp-api.onrender.com/";
}
```

## Step 2 – A model that matches the JSON

The API returns column names in snake_case (`is_done`, `created_at`). C# uses PascalCase, so we map them with `[JsonPropertyName]`. We call the class `TaskItem` because `Task` already means something in C# (async tasks).

`client/AppClient/Models/TaskItem.cs`

```
using System.Text.Json.Serialization;

namespace AppClient.Models;

public class TaskItem
{
    [JsonPropertyName("id")]         public int Id { get; set; }
    [JsonPropertyName("title")]      public string Title { get; set; } = "";
    [JsonPropertyName("is_done")]    public bool IsDone { get; set; }
    [JsonPropertyName("created_at")] public DateTime CreatedAt { get; set; }
}
```

## Step 3 – The ApiClient service

`client/AppClient/Services/ApiClient.cs`

```
using System.Net.Http.Json;
using AppClient.Helpers;
using AppClient.Models;

namespace AppClient.Services;

public static class ApiClient
{
    // Create ONE HttpClient for the whole app and reuse it.
    private static readonly HttpClient http = new()
    {
        BaseAddress = new Uri(AppConfig.ApiBaseUrl),
        Timeout = TimeSpan.FromSeconds(90)        // Render free plan may be waking up
    };

    public static async Task<List<TaskItem>> GetTasksAsync()
    {
        return await http.GetFromJsonAsync<List<TaskItem>>("api/tasks") ?? new();
    }

    public static async Task<TaskItem?> CreateTaskAsync(string title)
    {
        var response = await http.PostAsJsonAsync("api/tasks", new { title });
        response.EnsureSuccessStatusCode();       // throws if status is 4xx/5xx
        return await response.Content.ReadFromJsonAsync<TaskItem>();
    }

    public static async Task DeleteTaskAsync(int id)
    {
        var response = await http.DeleteAsync($"api/tasks/{id}");
        response.EnsureSuccessStatusCode();
    }
}
```

## Step 4 – Use it from a form

**client/AppClient/Forms/MainForm.cs (event handlers)**

```
private async void MainForm_Load(object? sender, EventArgs e)
{
    await LoadTasksAsync();
}

private async Task LoadTasksAsync()
{
    try
    {
        lblStatus.Text = "Loading...";
        dgvTasks.DataSource = await ApiClient.GetTasksAsync();
        lblStatus.Text = "Ready";
    }
    catch (Exception ex)
    {
        lblStatus.Text = "Could not connect";
        MessageBox.Show("Could not load tasks: " + ex.Message, "Error",
                        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private async void btnAdd_Click(object? sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(txtTitle.Text))
    {
        MessageBox.Show("Please type a title first.");
        return;
    }

    btnAdd.Enabled = false;                      // stop double clicks
    try
    {
        await ApiClient.CreateTaskAsync(txtTitle.Text.Trim());
        txtTitle.Clear();
        await LoadTasksAsync();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Could not save: " + ex.Message);
    }
    finally
    {
        btnAdd.Enabled = true;
    }
}
```

## Why async / await?

A network request can take seconds. With `await`, the window stays responsive while waiting; without it the app freezes and Windows shows “Not responding”. Rule of thumb: button and load event handlers may be `async void`; every other async method returns `Task` or `Task<T>` and ends with “Async”.

**CHECKPOINT** Add a task in the C# app, then run SELECT * FROM tasks; in DBeaver. The new row is there. You have a working full-stack app!

SECTION 11

# UI Libraries for C#

All install through NuGet. Licences change – always confirm on the library's website before submitting or selling your project. The same list is in docs/UI-LIBRARIES.md.

## Windows Forms

| Library | NuGet package | Licence | Best for |
|---|---|---|---|
| MaterialSkin.2 | `MaterialSkin.2` | MIT – free | Material Design look, easy to apply |
| ReaLTaiizor | `ReaLTaiizor` | MIT – free | Many themed controls, side menus, cards |
| Krypton Toolkit | `Krypton.Toolkit` | BSD-3 – free | Office-style controls, ribbon, docking |
| Guna UI 2 | `Guna.UI2.WinForms` | Commercial (trial) | Very modern rounded controls, shadows |
| Siticone UI | `Siticone.Desktop.UI` | Commercial (trial) | Modern controls similar to Guna |
| Bunifu UI | `Bunifu.UI.WinForms` | Commercial | Flat, animated controls |
| Syncfusion | `Syncfusion.*.WinForms` | Free Community Licence (if eligible) | Grids, charts, schedulers |
| DevExpress / Telerik | installer | Commercial (trial / student offers) | Full enterprise suites |

## WPF

| Library | NuGet package | Licence | Best for |
|---|---|---|---|
| WPF UI (lepoco) | `WPF-UI` | MIT – free | Windows 11 Fluent style, navigation |
| MaterialDesignInXamlToolkit | `MaterialDesignThemes` | MIT – free | Most popular Material library for WPF |
| MahApps.Metro | `MahApps.Metro` | MIT – free | Modern windows, flyouts, dialogs |
| HandyControl | `HandyControl` | MIT – free | 80+ extra controls and themes |
| ModernWpf | `ModernWpfUI` | MIT – free | Fluent styles (less active now) |

## Cross-platform C# UI

| Framework | Licence | Notes |
|---|---|---|
| Avalonia UI | MIT – free | WPF-like XAML on Windows, macOS, Linux. Themes: Material.Avalonia, FluentAvalonia. |
| .NET MAUI | MIT – free | One project for Windows, Android, iOS, macOS. |
| Uno Platform | Apache 2.0 – free | WinUI-style XAML on many platforms. |
| Blazor (web) | MIT – free | C# in the browser. UI kits: MudBlazor, Radzen Blazor, Fluent UI Blazor. |

## Helpers that work in WinForms and WPF

| Library | NuGet package | Use for |
|---|---|---|
| FontAwesome.Sharp | `FontAwesome.Sharp` | Icons on buttons and menus |
| LiveCharts2 | `LiveChartsCore.SkiaSharpView.WinForms` / `.WPF` | Animated charts and dashboards |
| ScottPlot | `ScottPlot.WinForms` / `ScottPlot.WPF` | Fast data plots |
| CommunityToolkit.Mvvm | `CommunityToolkit.Mvvm` | MVVM helpers (WPF, Avalonia, MAUI) |
| QuestPDF | `QuestPDF` | Printable PDF reports (community licence) |
| ClosedXML | `ClosedXML` | Export to Excel |

**TIP** Pick ONE framework and ONE main theme library. Mixing several theme libraries leads to controls that look and behave inconsistently. Beginner WinForms kit: MaterialSkin.2 (or ReaLTaiizor) + FontAwesome.Sharp. Beginner WPF kit: MaterialDesignThemes (or WPF-UI) + CommunityToolkit.Mvvm.

SECTION 12

# Working as a Team with Git and GitHub

Git lets several people change the same project without overwriting each other's work.

## The daily workflow

Terminal

```
git checkout main
git pull                                    # 1. get everyone's latest work
git checkout -b feature/task-list-screen    # 2. new branch for YOUR task

# ... write code, test it ...

git status                                  # 3. check what changed (no .env!)
git add .
git commit -m "Add task list screen"        # 4. save a snapshot
git push -u origin feature/task-list-screen  # 5. upload the branch
```

1. On GitHub, open a **Pull Request** (PR) from your branch into main.
2. A teammate reviews it, then clicks **Merge**.
3. Everyone runs git pull on main to get the change.

## Branch and commit naming

| Type | Branch example | Commit message example |
|---|---|---|
| New feature | `feature/login-form` | Add login form with validation |
| Bug fix | `fix/task-delete-404` | Fix 404 when deleting a task |
| Documentation | `docs/api-endpoints` | Document task endpoints |

## Avoiding merge conflicts

- Pull often and keep branches short-lived (merge within a few days).
- Split work by folder: frontend people in client/, backend people in server/.
- Do not let two people edit the same WinForms `.Designer.cs` file at once – designer files conflict badly. Assign each form to one person.
- If a conflict happens, open the file, keep the correct lines, delete the `<<<<<<<` / `=======` / `>>>>>>>` markers, then commit.

SECTION 13

# Security Basics

Even a school project should follow these. They are also what reviewers and teachers look for.

- [ ] **No secrets in Git.** `.env`, `ca.pem` and passwords stay out of the repository (already covered by .gitignore).
- [ ] **No secrets in the C# app.** The .exe only knows the API URL. Anything compiled into the app can be read by anyone.
- [ ] **Parameterised queries only.** `pool.query(sql, [values])` – never string concatenation.
- [ ] **Hash passwords** with `bcrypt` before saving. Compare with `bcrypt.compare` at login.
- [ ] **Validate input on the server**, even if the C# app already checks it. Anyone can call your API with Postman.
- [ ] **Return safe error messages.** Log the real error on the server; send the user a short, general message.
- [ ] **Protect private endpoints** with a login token (JWT) – see Section 14.
- [ ] **If a secret leaks**, rotate it immediately: reset the Aiven password and update Render + .env.

SECTION 14

# Next Step: Adding Login (bcrypt + JWT)

Do this only after the task example works end to end. This is an outline, not a full implementation.

Terminal (in server/)

```
npm install bcrypt jsonwebtoken
```

## How it works

1. **Register:** C# sends username + password → server hashes the password with `bcrypt.hash(password, 10)` → saves the hash in `users`.
2. **Login:** C# sends username + password → server finds the user → `bcrypt.compare(password, user.password_hash)` → if correct, creates a token with `jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: "1d" })` and returns it.
3. **C# keeps the token** in memory and sends it with every request.
4. **Server middleware** (`src/middleware/auth.js`) reads the `Authorization: Bearer <token>` header, checks it with `jwt.verify`, and sets `req.user`. If invalid, it returns 401.

server/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');

function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ message: 'Please log in' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Session expired, please log in again' });
  }
}

module.exports = requireAuth;
// usage in index.js:  app.use('/api/tasks', requireAuth, taskRoutes);
```

C# – send the token with every request

```
// after a successful login:
http.DefaultRequestHeaders.Authorization =
    new System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", token);
```

SECTION 15

# Troubleshooting

Read the error message carefully – it usually says exactly what is wrong. Then find it below.

| Problem / error message | Likely cause | Fix |
|---|---|---|
| `'node' / 'dotnet' / 'git' is not recognized` | Not installed, or terminal opened before installing | Install it, then close and reopen the terminal (or restart the PC). |
| `Cannot find module 'express'` | Packages not installed / wrong folder | Run `npm install` inside `server/`. |
| `ENOENT ... ca.pem` | Certificate file missing or wrong DB_CA_PATH | Put the Aiven certificate at `server/certs/ca.pem`; on Render check the secret file name. |
| `self-signed certificate in certificate chain` | `?sslmode=require` still in DATABASE_URL, or CA not loaded | Remove ?sslmode=require from the URL and make sure ca is passed in db.js. |
| `password authentication failed` | Wrong password / copied with extra spaces | Copy the Service URI again from Aiven. |
| `relation "tasks" does not exist` | Migration not run on this database | Run the migration SQL in DBeaver on the Aiven database. |
| `EADDRINUSE :::3000` | Server already running in another terminal | Close the other terminal or change PORT. |
| `req.body` is undefined | Missing `app.use(express.json())` or client did not send JSON | Add the line before the routes; in Postman choose Body → raw → JSON. |
| Render deploy fails: "Cannot find package.json" | Root Directory not set | Set Root Directory to `server`. |
| Render works, then is very slow first time | Free service was sleeping | Normal. Wait up to a minute; warm it up before demos. |
| `TaskCanceledException` in C# | Timeout (server waking up) or wrong URL | Check the URL in AppConfig; keep a 90-second timeout. |
| `HttpRequestException: 404` | Wrong path or missing trailing slash in BaseAddress | BaseAddress must end with `/`; paths must not start with `/`. |
| C# properties are empty / false after loading | JSON names don't match property names | Add `[JsonPropertyName("column_name")]` to the model. |
| Window freezes when clicking a button | Calling `.Result` or `.Wait()` on an async method | Use `async` / `await` all the way. |
| Designer file merge conflict | Two people edited the same form | Keep one version, redo the other person's change; assign forms to one owner. |

SECTION 16

# Final Checklist

Go through this list before you submit or present.

## Database

- [ ] All migrations are in database/migrations/ and numbered.
- [ ] Seed data exists for the demo.
- [ ] ERD image is in database/diagrams/.

## Server

- [ ] Every endpoint tested in Postman (success and error cases).
- [ ] No passwords in code; .env not in Git.
- [ ] All SQL uses $1, $2 placeholders.
- [ ] Live on Render; health URL returns ok.
- [ ] docs/API.md lists every endpoint.

## Client

- [ ] AppConfig points to the Render URL (not localhost).
- [ ] Every button shows loading/success/error feedback.
- [ ] App does not crash when the internet is off (try it!).
- [ ] Input is validated before sending.
- [ ] Consistent colours, fonts and control names.

## Project

- [ ] README files are up to date (how to run each part).
- [ ] All branches merged; main works from a fresh clone.
- [ ] Demo script rehearsed; Render warmed up 5 minutes before presenting.
- [ ] Release build of the C# app created (Build → Publish).

SECTION 17

# Glossary and Useful Links

| Term | Meaning |
|---|---|
| async / await | C# and JavaScript keywords to wait for slow work (network, database) without freezing. |
| CA certificate | A file that proves the database server is really Aiven's, so the connection cannot be intercepted. |
| Controller | Server function that handles one request and sends the response. |
| CORS | Browser rule about which websites may call your API. Not an issue for C# desktop apps. |
| ERD | Entity Relationship Diagram: a picture of your tables and how they connect. |
| Express | A small, popular web framework for Node.js. |
| HttpClient | The C# class for sending HTTP requests. |
| JWT | JSON Web Token: a signed string that proves a user is logged in. |
| Middleware | Express function that runs before/after routes (auth, logging, errors). |
| Migration | A versioned SQL script that changes the database structure. |
| Model | Server file with the SQL for one table (or, in C#, a class that holds data). |
| NuGet | The package manager for .NET (like npm for Node.js). |
| ORM | A library that writes SQL for you (e.g. Prisma, Sequelize). Not used in this guide so you learn real SQL first. |
| Pool | A set of reusable database connections. |
| REST | A style of API design using URLs for resources and HTTP methods for actions. |
| Route | Mapping of a URL + method to a controller function. |
| Seed | Sample data inserted for testing. |

## Official documentation

| Topic | Link |
|---|---|
| Express | expressjs.com |
| node-postgres (pg) | node-postgres.com |
| PostgreSQL tutorial | postgresql.org/docs/current/tutorial.html |
| Render – Node/Express deploy | render.com/docs/deploy-node-express-app |
| Render – environment variables & secret files | render.com/docs/configure-environment-variables |
| Aiven for PostgreSQL – getting started | aiven.io/docs/products/postgresql/get-started |
| C# for beginners | learn.microsoft.com/dotnet/csharp |
| Windows Forms | learn.microsoft.com/dotnet/desktop/winforms |

| Topic | Link |
| --- | --- |
| WPF | learn.microsoft.com/dotnet/desktop/wpf |
| HttpClient & JSON | learn.microsoft.com/dotnet/fundamentals/networking/http/httpclient |
| Git basics | git-scm.com/book |